Code and data for "Prediction-Driven Non-Preemptive Scheduling with Bounded Overtaking: A Per-Job Wait Guarantee"

Yazhou Guo, Zexin Lin, Chengyang Huo, Yurong Song. Manuscript mathematics-4625685, *Mathematics* (MDPI), 2026, first revision.

Version 1.1.0-revision. Archived at Zenodo, https://doi.org/10.5281/zenodo.23643805 (all versions: https://doi.org/10.5281/zenodo.23035899). The same files are in the GitHub repository https://github.com/oscar030406/spjf-guard under the tag `v1.1.0-revision`. Version 1.0.1-submitted (doi:10.5281/zenodo.23039931) is the package of the manuscript as first submitted.

This package holds the code, the configuration, the result tables and the run logs behind every number in the revised manuscript and its Supplementary Materials. It also holds the Lean 4 files that prove the main results of Section 6. Its code rebuilds those numbers from the public datasets that the paper uses. The package does not contain the datasets themselves (see "Data availability") or the manuscript.

The layout follows the replication-package template of the Social Science Data Editors (the AEA template README, v1.1). It covers data availability, computational requirements, a description of the programs, instructions, and a list of tables. The list names the programs and files that produce each table.

Contents

Path	What it holds
`README.md`, `README.pdf`	this file
`src/spjf_guard/`	the Python package that runs the main experiment: the queue simulator in front of `k` servers, which checks the paper's per-job bound on every simulated job (`sim/`); dataset readers and the lock on the sealed data (`data/`); causal features (`features/`); the running-time predictor (`predict/`); replay traces, parameter selection, metrics and tables (`experiment/`)
`scripts/`	the command-line steps of the pipeline, listed under "Instructions to replicators"
`configs/`	`main.yaml` holds every setting of the main experiment, each with a comment; the dated files are the configurations of the sensitivity runs, of the visibility runs and of the ACcoding held-out evaluation
`tests/`	unit and property tests of the package
`formal/`	Lean 4 proofs of the main results of Section 6, with the Lake project that pins Lean and Mathlib and the compiler output of two builds; `formal/README.md` gives the build commands and maps each paper result to its Lean theorem (Supplementary Section S12)
`evidence/`	the separate studies behind the numbers that the package does not produce (other workloads, the predictor comparison, numerical checks of the propositions, capacity and physical runs, and the experiments added in the revision), one folder per study with its scripts, its logs and a `README.md`; `evidence/README.md` lists them, and `evidence/revision_r1/README.md` lists the studies added in the revision
`outputs/`	the result tables that the revised manuscript reports, as the package wrote them; `outputs/README.md` says which command wrote each directory
`data/`	empty apart from `data/README.md`: where to download each dataset, where to save it, and the SHA-256 of the files we used

Path	What it holds
`docs/`	`sealed_run_procedure.md`: how the sealed semesters were run once; `sealed_access_log.md` (with an English rendering, `sealed_access_log.en.md`): every read of sealed data; `policy_consistent_visibility.md` (with an English rendering, `policy_consistent_visibility.en.md`): the exact visibility protocol, with its development results and run cost; `post_run_changes.json`: changes made after the freeze, each with its reason; `revision/accoding_sealed_analysis.md`: the timeline and checks of the run on ACcoding's sealed part; `adr/0008-online-replay-headline.md` and `adr/0009-zero-cost-blocks-leave-every-term.md`: the two design decisions that the paper cites
`prechecks/`	two modules that scripts import from these paths: `timeout_rule/timeout_overlays.py` (the online replay) and `accoding_v2/accoding_v2.py` (the ACcoding held-out runs; `protocol_lock_accoding.json` records its hash at this path)
`protocol_lock.json`, `protocol_lock.draft.json`, `protocol_lock_accoding.json`	`protocol_lock.json`: the hashes of code, configuration and inputs at the freeze of 2026-09-25, and the pinned choices (split, seeds, selected parameters). `protocol_lock.draft.json`: the same record computed from the code of this package. `protocol_lock_accoding.json`: the separate lock of the ACcoding held-out evaluation, frozen on 2026-10-03
`GENERATED.md`	every generated file: its source, the command that regenerates it and the command that checks it
`pyproject.toml`, `uv.lock`	the Python version and the locked dependency versions
`CITATION.cff`, `LICENSE`	how to cite; MIT licence for the code
`MANIFEST.sha256`	SHA-256 of every file in the package

Data availability and provenance

Statement about rights

The authors have legitimate access to and permission to use every dataset in this study. Third parties publish all of them, and all of them are publicly downloadable. None needs an access agreement. We redistribute none of them, as the manuscript's Data Availability Statement says. The package contains code, configuration, aggregate tables and run logs. It contains no dataset row, student identifier, user name or line of source code written by a student.

Datasets

Dataset	Provided here	Where to get it	Terms
CodeBench v1.81 (Universidade Federal do Amazonas), 18 semesters	no	https://codebench.icomp.ufam.edu.br/dataset/	no licence stated; cited, not redistributed
ACcoding v1.0.0 (Chen et al. 2024, doi:10.1038/s41597-024-03392-z)	no	doi:10.5281/zenodo.6522395	CC BY 4.0 per the data descriptor
OULAD (Kuzilek et al. 2017, doi:10.1038/sdata.2017.171)	no	https://analyse.kmi.open.ac.uk/open_dataset	CC BY 4.0
Azure Functions Invocation Trace 2021 (Zhang et al., SOSP 2021)	no	https://github.com/Azure/AzurePublicDataset	CC BY 4.0
Intel Netbatch 2012, Parallel Workloads Archive (Shai, Shmueli and Feitelson, JSSPP 2013)	no	https://www.cs.huji.ac.il/labs/parallel/workload/l_intel_netbatch/	free for research, with acknowledgement

Dataset	Provided here	Where to get it	Terms
LPC-EGEE 2004, Parallel Workloads Archive	no	https://www.cs.huji.ac.il/labs/parallel/workload/l_lpc/	free for research, with acknowledgement
Mozilla Firefox CI, two hardware worker pools, August–September 2026	no	collected from Mozilla's public Taskcluster and Treeherder APIs by `evidence/firefox_ci/run_collect.sh`	no licence stated; attributed to Mozilla

`data/README.md` gives the path at which each file must be saved and the checksums of the files that we used. It also gives the acknowledgements that the archives ask for.

Computational requirements

Software

Python 3.12 with the dependency versions locked in `uv.lock` (numpy, pandas, pyarrow, numba, LightGBM, and pytest, ruff and mypy for the checks), installed with uv. We made the reported runs on Windows 11 x86-64.

We also checked Linux with the same locked versions on 2026-09-25. On Linux, 714 of the 147,380 jobs of the first target semester get a different score from the refit of the frozen predictor. The difference is in the last bit. The runners that assert that the refit equals the released scores stop there. Everything else rebuilds.

The Lean proofs in `formal/` need elan, which installs the Lean version that `formal/lean-toolchain` names (`v4.35.0-rc4`). They also need Mathlib at the commit that `formal/lake-manifest.json` records. `formal/README.md` lists the disk, memory and network needs.

Controlled randomness

Every random draw uses numpy's `default_rng` with a fixed seed:

- the copy shifts of the replay traces: `overlay.seed` = 3 in `configs/main.yaml` (seed × 100 + overlay, `src/spjf_guard/experiment/overlay.py`), and seed × 100 + 50 for the single-server trace
- the predictor: `predictor.seed` = 3, with LightGBM's thread count fixed at 4, because the order in which it reduces its histograms depends on the thread count
- the paired week-block bootstrap: `bootstrap.seed` = 20260919
- the class-term bootstrap: `BOOT_SEED` = 20260924 in `scripts/run_cluster_bootstrap.py` (`scripts/run_online_cluster_bootstrap.py` reuses its draws)
- the audit samples and the timing sample: `scripts/run_online_visibility.py`, `scripts/run_consistent_visibility.py`, `selection_cost_seed` in `configs/main.yaml`
- the ACcoding held-out evaluation: the seeds in `configs/accoding_heldout_20260928.yaml`
- the studies in `evidence/`: each study's `README.md` names its seeds

Memory, runtime and storage

We measured these times on the authors' machine: Intel Core Ultra 9 275HX (24 threads), 64 GB of memory and Windows 11. The runs used two worker processes with four threads each.

Step	Wall-clock time
Parse one CodeBench semester archive	about 11 seconds
Build the parsed cache, 11 development semesters	about 5 minutes
Fit the ranking scores (`--repeat` checks two runs are identical)	about 7 minutes
Build the replay traces	about 2 minutes
Choose the guard's parameters, 258 candidates × 15 validation cells	about 7.3 hours
Main run, 15 cells, 29 policies	about 48 minutes

Step	Wall-clock time
Predictor metrics	about 11 minutes
Visibility comparison	about 35 minutes
Exact policy-specific visibility	about 4.3 hours
Online replay, 75 policy-cells	about 4.7 hours
Class-term bootstrap, 100 resamples, three worker processes	about 23 hours (first to last resample file)
Class-term bootstrap under the online replay, 100 resamples	505 jobs, run on two machines from 1 to 6 October 2026
ACcoding held-out evaluation of the sealed block	about 4 hours
Lean proofs: download the compiled Mathlib, then build	about 8 minutes, then about 1 minute

The development pipeline takes about 41 hours in all, 23 of them in the class-term bootstrap. The studies added in the revision (`evidence/revision_r1/`) took about 22 hours on this machine. Score quality took 4.5 h, backfilling 5.8 h, one to three servers 1.4 h and late reports 8.6 h. The overhead study took 1.7 h, including its wait for an idle machine. In addition, the time-limit and family runs of `evidence/limit_family_selection/` ran as 91 jobs on two machines from 3 to 5 October 2026.

The pipeline needs about 30 GB of free disk. Of this, the raw data take 8 GB, the derived traces 17 GB and the per-job files 3.5 GB. Each running cell also needs about 750 MB of temporary files. The Lean build needs about 12 GB more. One worker process of the online replay uses about 6 GB of memory.

Instructions to replicators

Run everything from the package root. The steps are in order. Each step reads only what the steps before it wrote.

1. **Install.** `uv sync --extra dev`, then set the command prefix that every step below uses:

```
export UV="env -u PYTHONHOME -u PYTHONPATH -u UV_INTERNAL__PYTHONHOME \
  NUMBA_NUM_THREADS=4 OMP_NUM_THREADS=4 uv run --no-sync"
```

2. **Check the installation without any data** (the minimal example): the unit and property tests build their inputs synthetically.

```
$UV python -m pytest -q -m "not slow and not crosscheck"
```

3. **Download the data** into the paths listed in `data/README.md` and compare the checksums. The ACcoding dumps then go through `evidence/accoding/parse_sql.py`.

4. **Build the inputs** from the development semesters (none of these reads a sealed one):

```
DEV=2018-1,2018-2,2019-1,2019-2,2020-ERE,2020-1,2020-2,2021-1,2021-2,2022-1,2022-2
SCORES=data/derived/package_ranking_scores/forward_scores.parquet
$UV python scripts/parse_archive.py --semesters $DEV
$UV python evidence/codebench_service/code_features.py --only $DEV
$UV python scripts/build_cache.py --pool development
$UV python scripts/fit_scores.py --repeat
$UV python scripts/build_overlays.py --pool primary --score-parquet $SCORES
$UV python scripts/build_overlays.py --pool validation --score-parquet $SCORES
$UV python scripts/build_overlays.py --single-server
```

5. **Choose the parameters** on the validation traces:

```
$UV python scripts/select_parameters.py --workers 2 --out-dir outputs/selection_v4
$UV python scripts/select_aging.py --workers 2
```

6. **Run the experiments** on the primary traces:

```
$UV python scripts/run_main.py --selection outputs/selection_v4/selected_parameters.csv \
    --out-dir outputs/dev_tables --workers 2
```

```
$UV python scripts/run_main.py --prefix k1 --reps 0 --levels 0 \
    --selection outputs/selection_v4/selected_parameters.csv --out-dir outputs/dev_tables/k1
$UV python scripts/eval_scores.py --pool primary
$UV python scripts/run_visibility.py --pool primary --workers 2
$UV python scripts/run_consistent_visibility.py --config configs/visibility_development_20260924.yaml \
    --pool primary --workers 2 --resume \
    --controls data/derived/package_ranking_scores/consistent_controls_20260924.npz
$UV python scripts/run_online_visibility.py --pool primary --workers 2 --resume --policies all
$UV python scripts/online_paired_differences.py
$UV python scripts/run_cluster_bootstrap.py --resamples 100 --workers 3
$UV python scripts/run_online_cluster_bootstrap.py --resamples 100 --workers 3
```

`run_consistent_visibility.py` builds its own controls file when it runs with `--build-controls-only`. `GENERATED.md` has that command, the sensitivity runs (`configs/sensitivity_*.yaml`) and the restricted reselection under the online replay (`scripts/select_online.py`). It also has the further online runs that the revision reports: the single-server and short-job score variants and the co-ending sensitivity (`scripts/build_coending_overlays.py`, `scripts/run_coending_online.py`).

7. **The sealed semesters** (2023-1, 2023-2, 2024-1) were run once, after the method was frozen on 2026-09-25 (`protocol_lock.json`). The code refuses to open them without the `--unseal` flag and a valid lock. `docs/sealed_run_procedure.md` lists the commands of that run. `docs/sealed_access_log.md` records every read.

8. **ACcoding's sealed part** (submission ids 80–100%) was run once, on 2026-10-03, under its own lock `protocol_lock_accoding.json`, written by `scripts/freeze_accoding.py`. The command is in `GENERATED.md`. `docs/revision/accoding_sealed_analysis.md` gives the timeline, including an earlier attempt that stopped before it produced any result.

9. **Emit the tables and compare** with the copies in `outputs/`:

```
$UV python scripts/sealed_dev_contrast.py
$UV python scripts/emit_paper_tables.py
$UV python scripts/emit_paper_tables.py --dev-exact-dir outputs/dev_consistent_visibility \
    --dev-online-dir outputs/dev_online_visibility --dev-cluster-dir outputs/cluster_bootstrap \
    --out-dir outputs/consistent_paper_tables
$UV python scripts/check_overlays.py
$UV python scripts/diff_dev_tables.py
```

A rerun writes into `outputs/`. Compare what it writes with this package's copy, for example with `MANIFEST.sha256`.

10. **Check the Lean proofs** (no data needed):

```
cd formal
lake exe cache get
lake build
```

`formal/README.md` gives the expected output: no error, and 49 axiom reports that list only the three standard axioms of Lean.

The studies in `evidence/` run separately. Each folder's `README.md` gives its question, the paper items that it supports, its inputs and the command that reruns it. In their scripts, absolute paths appear as `<repo-root>` (the package root) and `<cache-dir>` (a scratch directory). Set both before you rerun a study.

Reproducibility scope

- **Provided and rebuilt by the commands above:** every table and figure listed below, from the raw datasets. For comparison, the package holds the result files of the reported runs in `outputs/` and the logs of the separate studies in `evidence/`.
- **Not provided:** the raw datasets, the derived traces and per-job files, and the manuscript. The raw datasets come from third parties, and `data/README.md` says how to download them. The pipeline writes the derived traces and per-job files (about 20 GB), and the commands above rebuild them.
- **Sealed data:** the three sealed CodeBench semesters and ACcoding's sealed part were each evaluated once, under a frozen protocol. Rerunning those runs reproduces their numbers. Such a rerun is not a new test.

- **Proofs:** `formal/` checks the results of Section 6 that Supplementary Section S12 lists. The parts that it lists as proved only on paper are not machine-checked.
- **One check not rerunnable from this package:** two gates (pass-or-fail checks) of `evidence/revision_r1/backfill_baselines` compare its kernel with reference implementations of backfilling from a separate, unpublished study. Their outputs are kept in `evidence/revision_r1/backfill_baselines/out/`.
- **Platform:** the reported runs rebuild bit for bit on Windows x86-64 with the locked dependencies. On Linux, the frozen predictor's refit differs in the last bit for 714 of 147,380 jobs (see "Software" above). The runners that assert equality stop there.

List of tables and programs

"Result file" is the file in this package from which we read the printed numbers. "Program" is what writes that file. Rows marked *no computation* are definitions or illustrations.

Manuscript

Item	Result file	Program
Figure 1	*no computation* (three-job example)	
Figure 2	*no computation* (diagram of the method)	
Figure 3	*no computation* (diagram of the guard)	
Figure 4	`evidence/revision_r1/noise_dose/out/noise_dose_table.csv`	`evidence/revision_r1/noise_dose/`; drawn by `evidence/revision_r1/figures/make_noise_dose.py`
Figure 5	`evidence/weakness1_attack/capacity_curve.csv`; `evidence/limit_family_selection/limit/limit_summary.csv`; `outputs/dev_online_visibility/online_comparison.csv`	`evidence/weakness1_attack/`, `evidence/limit_family_selection/limit/`, `scripts/run_online_visibility.py`; drawn by `evidence/revision_r1/figures/make_bound_observed.py`
Figure 6	`evidence/weakness1_attack/capacity_curve.csv`; `outputs/dev_tables/main_table.csv`	`evidence/weakness1_attack/`, `scripts/run_main.py`; drawn by `evidence/revision_r1/figures/make_operating_region.py`
Table 1	*no computation* (comparison with the literature)	
Table 2	`evidence/predictor_neural/out_evaluate_core.txt`	`evidence/predictor_neural/`
Table 3	*no computation* (policy definitions)	
Table 4	*no computation* (notation)	
Table 5	`evidence/codebench_audit/out_tail_and_counts.txt`; `evidence/accoding_v2/out_accoding_v2.txt`; `evidence/codebench_service_v2/out_service_v2.txt`; `evidence/cross_domain/out_audit_azure.txt`, `out_audit_netbatch.txt`; `evidence/firefox_ci/out_SUMMARY.txt`; `evidence/oulad/out_load.txt`	the studies of the same names
Table 6	*no computation* (the data each stage used; `configs/main.yaml`, `protocol_lock.json`)	
Table 7	`outputs/consistent_paper_tables/tab_visibility.tex`, from `outputs/dev_online_visibility/online_comparison.csv` and `outputs/dev_consistent_visibility/exact_comparison.csv`	`scripts/emit_paper_tables.py`, from `scripts/run_online_visibility.py` and `scripts/run_consistent_visibility.py`
Table 8	`outputs/consistent_paper_tables/tab_visibility_sealed.tex`; `outputs/sealed_dev_contrast/contrast.csv`	the sealed run (`docs/sealed_run_procedure.md`); `scripts/sealed_dev_contrast.py`

Item	Result file	Program
Table 9	outputs/dev_tables/main_table.csv, paired_differences.csv; outputs/dev_visibility/visibility_waits_and_lag.csv; outputs/online_paired/online_paired_differences.csv	scripts/run_main.py; scripts/run_visibility.py; scripts/online_paired_differences.py
Table 10	evidence/timeout_rule/timeout_intervals.csv, out_timeout_overlays.txt; outputs/online_paired/online_clock_intervals.csv; outputs/dev_tables/main_table.csv; outputs/dev_visibility/visibility_comparison.csv	evidence/timeout_rule/; scripts/online_paired_differences.py; scripts/run_main.py; scripts/run_visibility.py
Table 11	evidence/revision_r1/backfill_baselines/out/backfill_table.csv, backfill_cells.csv, tab_backfill_body.tex	evidence/revision_r1/backfill_baselines/
Table 12	outputs/dev_tables/bound_checks.csv, identity_residuals.csv, k1/bound_checks.csv; evidence/revision_r1/small_k_shapes/out/bound_summary.csv; evidence/revision_r1/report_delay/out/summary.csv; evidence/timeout_rule/; evidence/envelope_bound/out_summarise_tight.txt	scripts/run_main.py; the studies of the same names
Table 13	CodeBench rows: outputs/dev_tables/main_table.csv, outputs/dev_predictor/predictor_metrics.csv; other rows: evidence/cross_domain/out_cross_domain_table.txt, evidence/accoding_v2/out_accoding_v2.txt, evidence/firefox_ci/out_SUMMARY.txt	scripts/run_main.py, scripts/eval_scores.py; the studies of the same names
Table 14	evidence/revision_r1/report_delay/out/summary.csv	evidence/revision_r1/report_delay/
Table 15	evidence/revision_r1/guard_overhead/out/table.md, measure.json	evidence/revision_r1/guard_overhead/

Supplementary Materials

Item	Result file	Program
Table S1	evidence/cross_domain/out_cross_domain_table.txt; evidence/firefox_ci/out_SUMMARY.txt	evidence/cross_domain/, evidence/firefox_ci/
Table S2	evidence/predictor_neural/out_evaluate_core.txt; evidence/codebench_service_v2/out_service_v2.txt; evidence/accoding_v2/out_accoding_v2.txt	the studies of the same names
Table S3	evidence/codebench_audit/out_tail_and_counts.txt; evidence/accoding_v2/out_accoding_v2.txt; evidence/cross_domain/out_cross_domain_table.txt; evidence/firefox_ci/out_SUMMARY.txt	the studies of the same names
Table S4	outputs/paper_tables/tab_setup.tex, from outputs/dev_tables/; trace rows from evidence/main_v3/out_main_v31.txt	scripts/emit_paper_tables.py
Table S5	protocol_lock.draft.json (selected_parameters, family_best), which outputs/selection_v4/ must match	scripts/select_parameters.py; scripts/make_protocol_lock.py
Table S6	outputs/paper_tables/tab_guard.tex, from outputs/dev_tables/main_table.csv, policy_parameters.csv	scripts/emit_paper_tables.py from scripts/run_main.py
Table S7	outputs/paper_tables/tab_guard.tex (Guard-age and Guard-queue rows)	the same

Item	Result file	Program
Table S8	outputs/paper_tables/tab_adv.tex, from outputs/dev_tables/main_table.csv	the same
Table S9	outputs/paper_tables/tab_resid.tex, from outputs/dev_tables/identity_residuals.csv	the same
Table S10	outputs/paper_tables/tab_k1.tex, from outputs/dev_tables/k1/main_table.csv	the same
Table S11	outputs/selection_aging/aging_protocol.json, selected_aging.csv	scripts/select_aging.py
Table S12	outputs/dev_visibility/visibility_comparison.csv	scripts/run_visibility.py
Table S13	outputs/dev_predictor/predictor_metrics.csv; evidence/codebench_service_v2/out_service_v2.txt	scripts/eval_scores.py; evidence/codebench_service_v2/
Table S14	evidence/ranking_score/out_table_primary_5reps.txt	evidence/ranking_score/
Table S15	evidence/codebench_service_v2/out_service_v2.txt (SUMMARY 5)	evidence/codebench_service_v2/
Table S16	outputs/paper_tables/tab_rank.tex, from outputs/dev_tables/main_table.csv	scripts/emit_paper_tables.py from scripts/run_main.py
Table S17	outputs/selection_v3/selection_protocol.json	scripts/select_parameters.py
Table S18	outputs/consistent_paper_tables/tab_cluster.tex, from outputs/cluster_bootstrap/bootstrap_summary.csv	scripts/run_cluster_bootstrap.py, scripts/emit_paper_tables.py
Table S19	evidence/weakness1_attack/capacity_curve.csv, summary_numbers.json	evidence/weakness1_attack/
Table S20	evidence/weakness1_attack/physical_numbers.json, physical_verification.json	evidence/weakness1_attack/
Table S21	evidence/closed_loop/out/open_vs_closed_mean.csv, report_tables.md	evidence/closed_loop/make_tables.py
Table S22	evidence/lpc_egee_queues/tables.md	evidence/lpc_egee_queues/
Table S23	evidence/lpc_egee_queues/tables.md, policy_metrics.csv	evidence/lpc_egee_queues/
Table S24	evidence/lpc_egee/applicability.csv; evidence/lpc_egee_queues/tables.md	evidence/lpc_egee/, evidence/lpc_egee_queues/
Table S25	evidence/platform_testbed/records/summary.json, simulator_replay.json	evidence/platform_testbed/
Table S26	*no computation* (comparison with the literature; evidence/guard_optimality_verify/literature_check.md)	
Tables S27–S30	outputs/consistent_paper_tables/tab_rank_sealed.tex, tab_guard_sealed.tex, tab_adv_sealed.tex, tab_k1_sealed.tex, from outputs/sealed_tables/	the sealed run; scripts/emit_paper_tables.py
Table S31	outputs/consistent_paper_tables/tab_online_suite.tex, from outputs/dev_online_visibility/	scripts/emit_paper_tables.py from scripts/run_online_visibility.py
Table S32	outputs/online_paired/tab_s_online_pairs_body.tex	scripts/online_paired_differences.py
Table S33	outputs/online_paired/tab_s_online_clock_body.tex	scripts/online_paired_differences.py
Table S34	formal/README.md (the map of paper results to Lean theorems); formal/compile_output.txt, Guard_compile_output.txt (the axiom reports)	formal/ (lake build)
Table S35	evidence/revision_r1/small_k_shapes/out/tab_s_small_k_body.tex, guidance.csv, shape_table.csv	evidence/revision_r1/small_k_shapes/

Item	Result file	Program
Table S36	evidence/revision_r1/accoding_sealed/out/tab_s_accoding_sealed_body.tex, from outputs/accoding_heldout_sealed_online_v2/summary.csv	scripts/run_accoding_heldout.py; evidence/revision_r1/accoding_sealed/make_tex_table.py
Table S37	evidence/revision_r1/abandonment/out/tab_s_abandon_body.tex, from evidence/abandonment_loop/out/cells.csv	evidence/abandonment_loop/; evidence/revision_r1/abandonment/make_tex_table.py
Table S38	evidence/revision_r1/online_cluster/out/tab_s_online_cluster_body.tex, from outputs/online_cluster_bootstrap/bootstrap_summary.csv, outputs/dev_online_visibility/online_comparison.csv and outputs/online_paired/online_paired_differences.csv	scripts/run_online_cluster_bootstrap.py; evidence/revision_r1/online_cluster/make_tex_table.py
Table S39	evidence/revision_r1/sealed_online_cluster/out/tab_s_sealed_online_cluster_body.tex, from outputs/sealed_online_cluster_bootstrap/bootstrap_summary.csv	scripts/run_online_cluster_bootstrap.py --pool sealed --unseal; evidence/revision_r1/sealed_online_cluster/make_tex_table.py
Figure S1	outputs/dev_tables/main_table.csv; outputs/dev_visibility/visibility_comparison.csv; outputs/selection_v4/selected_parameters.csv; outputs/selection_aging/selected_aging.csv	scripts/emit_gap_harm_figure.py
Figure S2	outputs/paper_tables/tab_guard.tex	drawn from the table; no new computation
Figure S3	outputs/paper_tables/	drawn from the tables; no new computation

Propositions checked numerically or by machine

Paper item	Study
Section 6: the identity, the guard bound, the necessity of a work budget and the price of the guarantee	formal/ (Lean 4; Supplementary Section S12 lists what it covers)
Section 6 and Supplementary S4, S8: the identity, the guard bound, tightness	evidence/guard_theory/
Proposition 1 (maximum waiting time) and Table 10	evidence/timeout_rule/, evidence/new_theory_refutation/
Proposition 2 (absolute bound) and Section 8.5	evidence/envelope_bound/
Proposition 3 (late completion reports) and Section 8.9	evidence/stale_charging/, evidence/revision_r1/report_delay/
The cost ratio 150 (Sections 1, 6.4, 10)	evidence/cost_ratio/
Supplementary S7.1–S7.4	evidence/guard_optimality_verify/
Supplementary S7.3, S7.7, S7.8 (Propositions S8, S17–S19)	evidence/general_reference/
Supplementary S7.6	evidence/reservation_guard/, evidence/reservation_guard_verify/, evidence/public_cap_guard/

evidence/README.md maps every other sentence-level number to its study and log line.

What differs from the authors' working repository

The package is the revision commit (3375919) with the changes below. We changed nothing else in the commit.

We left out these files:

- the manuscript sources and the Chinese copies of the documents
- development tooling
- five studies that the paper does not use and that no packaged script reads (evidence/README.md names them)

- from the other studies, what the paper does not cite: working notes (plans, reviews, audits), development checks (smoke and gate runs) and per-run logs
- the design records other than ADRs 0008 and 0009, and the glossary of the working repository, which some documents still name
- the response letters and working notes of the revision
- the notes of a separate theory study
- two notes in Chinese that compare each Lean statement with the paper's

We added these files:

- `outputs/`, the result tables, which the repository does not track
- this README, `data/README.md`, `outputs/README.md` and `MANIFEST.sha256`
- `evidence/accoding/parse_sql.py` and a short `evidence/accoding/README.md`. The parser turns the SQL dumps of ACcoding into the tables that `evidence/accoding_v2/`reads. In the parser, `<repo-root>` replaces the absolute paths.

We edited these files:

- the first lines of `GENERATED.md` and `docs/sealed_run_procedure.md`, which pointed to the Chinese originals
- the `accoding` row of `evidence/README.md`
- the row of `formal/README.md` that named the notes in Chinese
- the two compiler outputs of `formal/` from a Linux run: we removed the server path and kept the file names
- `CITATION.cff`, which now also gives this version and its DOI

The configuration files still mention `data/derived/README.md`, which this package does not include. We left them as they are, because the protocol lock and every `outputs/*/manifest.json` record their hashes.

We checked the tables in `outputs/` against the revised manuscript on 2026-10-26, in a copy of this package with its own environment. With the manuscript source given as `--paper`, `scripts/check_paper_numbers.py` reports that every number that the paper prints is the one that the package produced. The tests of step 2 gave 392 passed, 8 skipped and 11 deselected (the slow and cross-check tests). Seven of the skipped tests compare tables with the manuscript source, which this package does not include. The eighth needs a git checkout.

Licence and citation

The code is released under the MIT licence (`LICENSE`). The licence does not cover the datasets. Each dataset keeps its publisher's terms. To cite this package, cite the article and this archive. The article: Guo, Y.; Lin, Z.; Huo, C.; Song, Y. Prediction-Driven Non-Preemptive Scheduling with Bounded Overtaking: A Per-Job Wait Guarantee. Mathematics 2026, 14, 4020, doi:10.3390/math14214020. The archive: Guo, Y.; Lin, Z.; Huo, C.; Song, Y. Code and data for "Prediction-Driven Non-Preemptive Scheduling with Bounded Overtaking: A Per-Job Wait Guarantee", version 1.1.0-revision. Zenodo, 2026, doi:10.5281/zenodo.23643805. `CITATION.cff` gives the same in machine-readable form.